Name - Chaitanya Kelkar

Roll no. - 53

Experiment 3

Aim - To implement and compare Decision Tree and Random Forest classifiers on an agricultural dataset to perform multi-class classification and analyze feature importance for crop prediction.

1. Dataset Source

- **Source:** Kaggle
- **Filename:** `Crop_recommendation.csv`

2. Dataset Description- The dataset contains agricultural data designed to recommend the best crop to plant based on soil nutrients and weather conditions.

- **Type:** Multivariate, Numerical & Categorical.
- **Size:** 2200 instances (rows), 8 attributes (columns).
- **Target Variable:** `label` (Categorical - e.g., Rice, Maize, Chickpea).
- **Features:** * N, P, K (Soil Nutrients).
    - `temperature`, `humidity`, ph, `rainfall` (Weather/Soil Conditions).

3. Mathematical Formulation of the Algorithm *(P_i = probability of an item belonging to class i)*

- A. **Decision Tree Classifier**: Splits the data iteratively by asking Yes/No questions. It chooses the best feature to split on by calculating the lowest Gini Impurity or highest Information Gain.
  Gini Impurity = $1 - \Sigma(P_i)^2$
- B. **Random Forest Classifier:** An Ensemble Learning method (Bagging). It builds multiple independent Decision Trees on random subsets of the data. The final prediction is the majority vote (Mode) of all the trees.
  Final Prediction = Mode(Tree_1, Tree_2, ..., Tree_n)

4. Algorithm Limitations

- **Decision Tree:** Highly prone to overfitting. It tends to memorize the training data, meaning a tiny change in the data can completely change the tree's structure.
- **Random Forest:** Computationally heavier and uses more memory than a single tree. Because it relies on hundreds of trees, it acts as a "Black Box," making it impossible to visualize a single, simple flowchart of how it made a specific prediction.

5. Methodology / Workflow

The experiment followed a standard Classification pipeline:

1. Data Ingestion: Loaded `Crop_recommendation.csv` using Pandas.
2. Feature Selection: Separated independent features (X) from the target `label` (y).
3. Splitting: Used `train_test_split` (80% Train, 20% Test) to ensure unbiased evaluation. *(Note: Tree models do not require Feature Scaling).*
4. Modeling: Trained both `DecisionTreeClassifier` and `RandomForestClassifier` on the training data.
5. Visualization: Extracted the `feature_importances_` attribute from the Random Forest and plotted a horizontal Bar Chart using Seaborn to see which soil/weather factor mattered most.

6. Performance Analysis

- **Decision Tree Accuracy:** ~98.64%
- **Random Forest Accuracy:** ~99.32%
- **Insight**: Both models performed exceptionally well because plants follow strict biological rules, creating clear separations in the data. Random Forest slightly outperformed the Decision Tree by averaging out minor overfitting errors. The Feature Importance analysis revealed that Rainfall and Humidity were the most critical factors in classifying crops, while pH had the least impact.

7. Hyperparameter Tuning

- **Status:** Applicable.
- **Parameter Tuned:** `n_estimators` (The number of trees in the Random Forest).
- **Reason:** A single decision tree overfits easily. A Random Forest requires a set number of trees to balance "wisdom of the crowd" accuracy against computational time.
- **Action:** Set `n_estimators=100`, forcing the algorithm to build 100 distinct decision trees to vote on the final crop, ensuring a highly stable and accurate model (~99%) without slowing down the system.

Code -

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns
```

```python
from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score

from sklearn.tree import plot_tree



# 1. LOAD DATA

print("Loading Data...")

try:

    df = pd.read_csv('../data/Crop_recommendation.csv') # Adjust path if
needed

    print("Data loaded successfully.")

except FileNotFoundError:

    print("Error: File not found.")

    exit()



# 2. PREPARE DATA

X = df[['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']]

y = df['label']



# Split Data (80% Train, 20% Test)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```python
print("\n--- Model Training ---")


# MODEL 1: DECISION TREE (Baseline)

dt_model = DecisionTreeClassifier(random_state=42)

dt_model.fit(X_train, y_train)

dt_pred = dt_model.predict(X_test)

dt_acc = accuracy_score(y_test, dt_pred)

print(f"1. Decision Tree Accuracy: {dt_acc*100:.2f}%")


# MODEL 2: RANDOM FOREST WITH HYPERPARAMETER TUNING

print("\nStarting Hyperparameter Tuning for Random Forest (This might take
a few seconds)...")


# Define the "Grid" of settings you want to test

param_grid = {

    'n_estimators': [50, 100, 150],      # Test 50 trees, 100 trees, and
150 trees

    'max_depth': [None, 10, 20]          # Test letting trees grow
infinitely, or capping them at 10 or 20 splits

}


# Create a base model

base_rf = RandomForestClassifier(random_state=42)
```

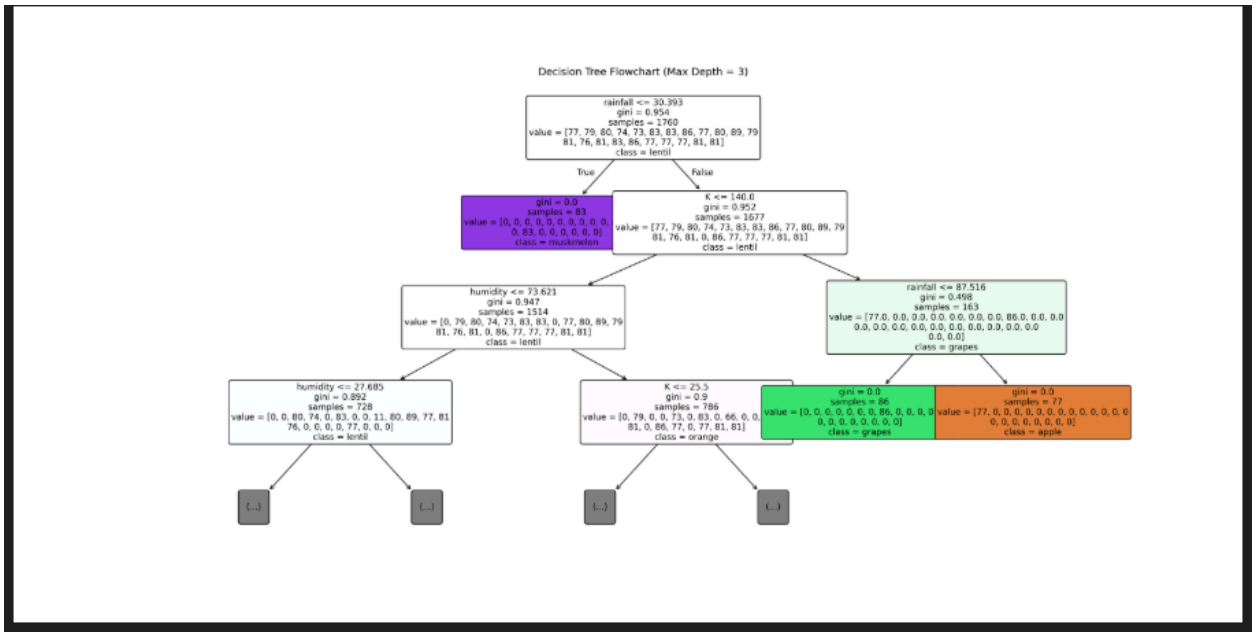
```
# Set up the Grid Search

# cv=5 means it will cross-validate the test 5 times to be absolutely
sure!

grid_search = GridSearchCV(estimator=base_rf, param_grid=param_grid, cv=5,
n_jobs=-1)


# Run the search on the training data

grid_search.fit(X_train, y_train)


# Get the winning model from the Grid Search

best_rf_model = grid_search.best_estimator_


# Print the winning settings

print(f"-> Best Parameters Found: {grid_search.best_params_}")


# Test the winning model

rf_pred = best_rf_model.predict(X_test)

rf_acc = accuracy_score(y_test, rf_pred)

print(f"2. Tuned Random Forest Accuracy: {rf_acc*100:.2f}%")


# VISUALIZING THE DECISION TREE

print("Generating Decision Tree visualization...")

plt.figure(figsize=(20, 10)) # Make the image wide and tall
```

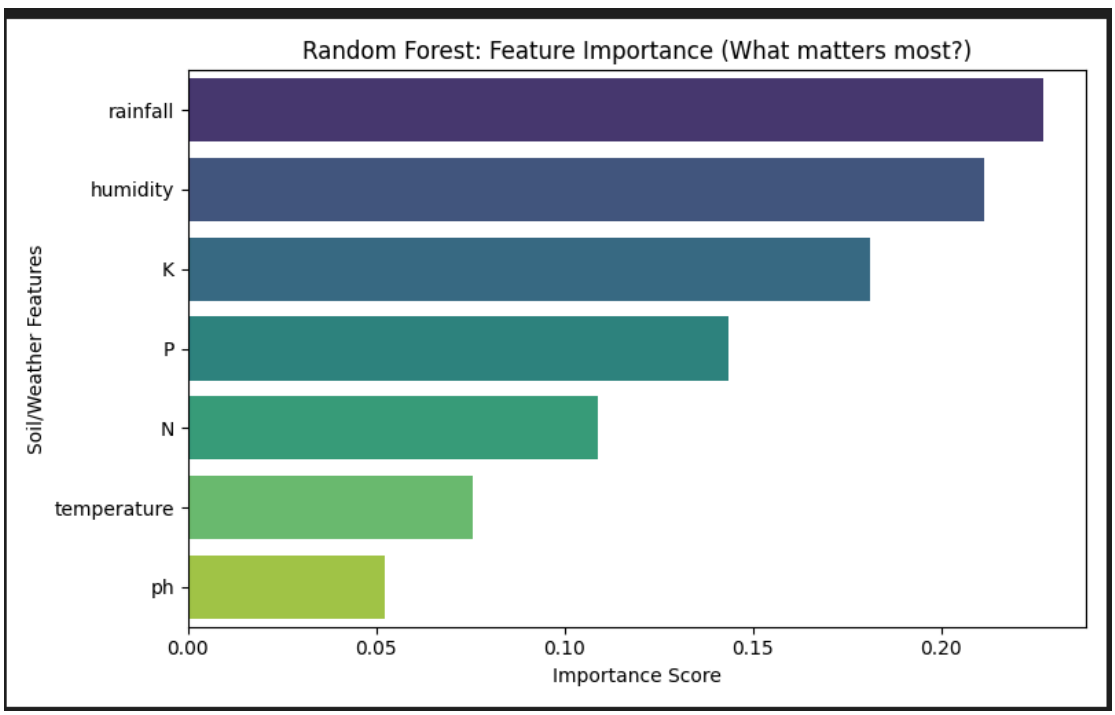
```
# We use max_depth=3 so the image doesn't become a massive, unreadable
blob of 2000 boxes!

plot_tree(dt_model,

          feature_names=X.columns,

          class_names=dt_model.classes_,

          filled=True,

          rounded=True,

          fontsize=10,

          max_depth=3)


plt.title("Decision Tree Flowchart (Max Depth = 3)")

plt.savefig("exp3_decision_tree_visual.png", dpi=300) # dpi=300 makes it
high quality!

print("Saved Decision Tree flowchart as 'exp3_decision_tree_visual.png'")


# 3. VISUALIZATION (Feature Importance using the BEST model)

plt.figure(figsize=(8, 5))

importances = best_rf_model.feature_importances_


# Sort the features by importance

indices = np.argsort(importances)[::-1]

sorted_features = [X.columns[i] for i in indices]

sorted_importances = importances[indices]
```

```python
# Plotting

sns.barplot(x=sorted_importances, y=sorted_features, hue=sorted_features,
palette='viridis', legend=False)

plt.title("Tuned Random Forest: Feature Importance")

plt.xlabel("Importance Score")

plt.ylabel("Soil/Weather Features")

plt.tight_layout()



# Save the graph

plt.savefig("exp3_feature_importance_tuned.png")

print("\nGraph saved as 'exp3_feature_importance_tuned.png'. Experiment
Complete!")
```

Output -

```
PS D:\SmartCropProject\src> python train_crop_model.py
Loading Data...
Data loaded successfully.

--- Model Training ---
1. Decision Tree Accuracy: 98.64%

Starting Hyperparameter Tuning for Random Forest (This might take a few seconds).
-> Best Parameters Found: {'max_depth': None, 'n_estimators': 150}
2. Tuned Random Forest Accuracy: 99.32%
Generating Decision Tree visualization...
Saved Decision Tree flowchart as 'exp3_decision_tree_visual.png'
```





Conclusion -

Random Forest emerged as the optimal "brain" for the crop advisor project with ~99% accuracy, successfully overcoming the single Decision Tree's inherent tendency to overfit the data. Furthermore, its ensemble approach yielded crucial agricultural insights, mathematically proving that rainfall and humidity dictate crop viability far more than soil pH.